Email Spam Detector with Self-Learning Feedback

Final Project Submission Documentation

1. Project Overview

This project is an end-to-end Machine Learning web application designed to classify email text as either *Spam* or *Legitimate (Ham)*. It was built to demonstrate a complete ML lifecycle, from data ingestion and model training to web deployment and continuous self-learning.

2. Architecture & Technologies

- **Backend Framework:** Flask (Python)
- **Machine Learning Library:** Scikit-Learn
- **Algorithm:** Random Forest Classifier
- **Feature Extraction:** TF-IDF (Term Frequency-Inverse Document Frequency)
- **Deployment & CI/CD:** Docker, GitHub Actions
- **Testing:** Pytest (100% Pass Rate across 58 unit tests)

3. Dataset & Model Performance

The model was recently upgraded to train on a real-world dataset comprising over **5,000 email samples**. This massive increase in training data significantly improved the robustness and generalizability of the model.

- **Total Samples:** 5,764
- **Accuracy:** 97.22%
- **Precision:** 93.25%
- **Recall:** 87.86%
- **F1-Score:** 0.9048

4. Key Engineering Features

1. **Explainable AI (XAI):** The application doesn't just predict "Spam"; it exposes the underlying `feature_importances_` array to highlight the exact words (features) that contributed most heavily to the model's decision.
2. **Self-Learning Loop (MLOps):** Users can provide feedback on predictions. If the model misclassifies an email, the user's correction is saved. A dedicated `/retrain` endpoint dynamically re-ingests this feedback, combining it with the base dataset to retrain the model and vectorizer on the fly.
3. **Robust Error Handling:** Comprehensive input validation and structured logging ensure the application handles malformed data gracefully without crashing.
4. **Containerization:** The entire application, including its dependencies, is containerized using Docker, ensuring perfectly reproducible environments across any system.

5. Conclusion

This project successfully demonstrates advanced software engineering and applied machine learning capabilities, making it a highly production-ready portfolio asset.

Email Spam Detector

Explain Like a Kid Version!

What is this?

Imagine you have a super smart robot guard standing by your mailbox. Every time an email tries to get in, the robot reads it very fast to figure out if it is a nice letter from a friend, or if it is "Spam" (a bad, tricky email from strangers trying to steal your allowance).

How does it work?

1. **The Dictionary:** The robot doesn't know English naturally! So, we gave it a giant dictionary of words. It counts how many times certain words appear.
2. **The Detective:** The robot looks at the words and acts like a detective. If it sees the word "FREE" or "WINNER" or "MONEY" a lot, it gets very suspicious!
3. **The Score:** The robot gives the email a score (like a grade in school). If it's 99% sure it's Spam, it throws it in the trash bin! If it's only 40% sure, it might ask you for help.

What happens if the robot makes a mistake?

Sometimes the robot gets confused. Maybe your grandma sent you an email that said "FREE COOKIES" and the robot threw it away!

Oh no! But don't worry, this robot is a **learning robot**. You can go to the robot and say, "Hey! That was a real email!" The robot says "I'm sorry!" and adds that to its brain. The next time grandma sends you an email about free cookies, the robot will remember and let it through.

Is it a good robot?

Yes! We tested it with almost 6,000 emails, and it got an A+ on its test (97.22% correct)!

Email Spam Detector

Student Learning Version

The Goal

The objective of this project is to build a machine learning model that can read an email and classify it as "Spam" or "Ham" (Not Spam). We also want to wrap this model in a web interface so normal users can interact with it.

Step 1: Preparing the Data (TF-IDF)

Computers can't read words like humans do; they only understand numbers. So, we have to convert the text into numbers using **TF-IDF**.

- **TF (Term Frequency):** How often does a word appear in *this* specific email?
- **IDF (Inverse Document Frequency):** How often does this word appear across *all* emails in the dataset? If a word like "the" appears everywhere, it's not useful. If a word like "Winner" appears only in a few emails, it's very useful!
  TF-IDF multiplies these together to score every word.

Step 2: The Model (Random Forest)

We use a **Random Forest Classifier**. Imagine a single "Decision Tree" as a flowchart. It asks: *Does the email contain "Money"? If yes, is it written in all capital letters?*
A single tree might make mistakes, so a Random Forest builds *hundreds* of these trees and makes them vote. If 90 out of 100 trees vote "Spam", the model predicts "Spam" with 90% confidence!

Step 3: Explainability (XAI)

Nobody likes a "black box" AI. We want to know *why* the model made its choice. Scikit-Learn provides `feature_importances_`. We map these importance scores back to the words they represent. This allows the web app to show the user exactly which words triggered the spam filter!

Step 4: The Web App (Flask)

We built a web server using **Flask**. It creates an endpoint `/check` where the frontend sends the email text. Flask hands the text to the Vectorizer, then the Model, and sends the final prediction back to the user's browser.

Step 5: Continuous Learning

Machine learning models get stale over time as spammers invent new tricks. We added a `/feedback` endpoint. If the model gets it wrong, the user can click "This is actually Spam!". We save this to a JSON file. The `/retrain` endpoint takes the original dataset, adds the new user feedback, and retrains the model from scratch!